

DPG确定性策略梯度法和LQR控制的联系

📖 DPG和LQR简化版

1. DPG算法的动作价值 $Q(x, u)$ 确定为LQR中的价值函数J时的情况，策略为状态反馈 $u = -Kx$ 的情况时

对于离散时间线性系统，在第 k 个控制采样时间点，状态为 x_k ，控制输入为 u_k （强化学习里的 action），系统动态为：

$$x_{k+1} = Ax_k + Bu_k$$

LQR 的目标是最小化二次型代价函数（是一个天然的李雅普诺夫函数）：

$$J = \sum_{k=0}^{\infty} (x_k^T Q x_k + u_k^T R u_k)$$

其中 $Q \succeq 0$ ， $R \succ 0$ 。

1. LQR 的最优解与 Q 函数 📖 离散系统的LQR最优控制推导

LQR 的最优控制律为状态反馈 $u_k = -Kx_k$ ，增益矩阵 K 通过求解离散代数 Riccati 方程得到：

$$P = Q + A^T P A - A^T P B (R + B^T P B)^{-1} B^T P A$$

最优增益为：

$$K = (R + B^T P B)^{-1} B^T P A$$

在k+1时刻的最优状态值函数为 $V^*(x_{k+1}) = x_{k+1}^T P x_{k+1} = (Ax_k + Bu_k)^T P (Ax_k + Bu_k)$ 。

在k时刻的最优动作值函数（Q 函数，即最优情况下的 J 函数）为：

$$Q^*(x_k, u_k) = x_k^T Q x_k + u_k^T R u_k + V^*(x_{k+1}) = x_k^T Q x_k + u_k^T R u_k + (Ax_k + Bu_k)^T P (Ax_k + Bu_k)$$

可整理为二次型形式：

$$Q^*(x_k, u_k) = \begin{bmatrix} x_k \\ u_k \end{bmatrix}^T H \begin{bmatrix} x_k \\ u_k \end{bmatrix}, \quad H = \begin{bmatrix} Q + A^T P A & A^T P B \\ B^T P A & R + B^T P B \end{bmatrix}。这个 $Q^*(x_k, u_k)$ 对 u_k 求$$

导，可以展开求，很简单，注意求偏导时 x_k 视为常数。

2. DPG 框架与 LQR Q 函数的结合

在 DPG 中，我们优化确定性策略 $\mu_\theta(x_k)$ ，参数为 θ 。目标是最小化期望累积代价，等价于最大化负代价的期望。

关键设定：

- 将 DPG 的 Q 函数近似器固定为 LQR 的最优 Q 函数，即 $Q^w(x_k, u_k) = Q^*(x_k, u_k)$ 。
- 策略参数化为线性形式： $\mu_K(x_k) = -Kx_k$ ，其中 $\theta = K$ 为待优化矩阵。

3. 策略梯度更新与收敛性

DPG 的策略梯度公式为：

$$\nabla_K J(K) = \mathbb{E}_{x_k \sim \rho^\mu} \left[\nabla_K \mu_K(x_k) \nabla_u Q^*(x_k, u) \Big|_{u=\mu_K(x_k)} \right]$$

其中 ρ^μ 为状态分布。

计算梯度分量：

- $\nabla_K \mu_K(x_k) = -x_k^\top$ （按分母布局，梯度矩阵维度为 $m \times n$ ，实际计算中使用外积形式）。
- $\nabla_u Q^*(x_k, u) = 2Ru + 2B^\top P(Ax_k + Bu)$ 。

代入 $u = -Kx_k$ ：

$$\nabla_u Q^*(x_k, -Kx_k) = -2(RK + B^\top P(A - BK))x_k$$

因此策略梯度为：

$$\nabla_K J(K) = -\mathbb{E} [\nabla_u Q^*(x_k, -Kx_k) x_k^\top] = 2\mathbb{E} [(RK + B^\top P(A - BK))x_k x_k^\top] \quad (\text{这个梯度和书上的顺序相反，是矩阵求导表示方式不同导致的})$$

假设状态探索充分，使得 $\mathbb{E}[x_k x_k^\top] = S \succ 0$ （正定），则梯度为零当且仅当：

$$(RK + B^\top P(A - BK))S = 0 \quad \Rightarrow \quad RK + B^\top P(A - BK) = 0$$

解得：

$$K = (R + B^\top P B)^{-1} B^\top P A$$

这与 LQR 的最优增益完全相同。

4. 结论

在以下条件下：

1. DPG 的 Q 函数精确等于 LQR 的最优 Q 函数 $Q^*(x_k, u_k)$ ；
2. 策略参数化为线性形式 $u_k = -Kx_k$ ；
3. 状态分布探索充分，使得 $\mathbb{E}[x_k x_k^\top]$ 正定；

DPG 的策略梯度更新会收敛到 LQR 的最优解。此时，策略梯度更新的静止点就是离散时间代数 Riccati 方程的解对应的增益矩阵。

注意：实际 DPG 算法中通常需要同时学习 Q 函数和策略。若 Q 函数通过函数近似学习，需保证其收敛到真实 Q^* ，否则策略收敛可能偏离 LQR 解。但在理论设定下，两者具有严格的一致性。

代码块

```
1 % dpg_lqr_convergence.m
2 % Q函数采用LQR的价值函数时，验证 DPG 策略梯度在真实 Q* 下收敛到 LQR 最优解
```

```

3  clear; clc; close all;
4  %% 1. 定义系统动态与代价矩阵 (Discrete-Time Linear System)
5  %  $x_{k+1} = A x_k + B u_k$ 
6  A = [1.1, 0.5;
7       0.0, 0.9];
8  B = [0.1;
9       1.0];
10 % 代价函数矩阵  $J = x'Qx + u'Ru$ 
11 Q = eye(2);          % 状态代价矩阵 (Q 必须半正定)
12 R = 1;               % 控制代价矩阵 (R 必须正定)
13 %% 2. 求解真实的 LQR 最优解 (作为 Baseline)
14 % 使用 MATLAB 内置的 DARE (Discrete Algebraic Riccati Equation) 求解
15 [P_star, ~, K_lqr] = dare(A, B, Q, R);
16 fprintf('--- LQR 最优解 (解析解) ---\n');
17 disp('最优反馈增益 K_lqr:');
18 disp(K_lqr);
19 %% 3. DPG 策略参数初始化与超参数设定
20 rng(42);              % 固定随机种子以保证结果可复现
21 K_dpg = rand(size(K_lqr)); % 随机初始化策略矩阵 K
22 learning_rate = 0.01; % 梯度下降学习率
23 num_iterations = 800;  % 迭代次数
24 batch_size = 200;     % 每次计算梯度的采样状态数 (保证状态探索充分)
25 % 记录误差用于绘图
26 error_history = zeros(num_iterations, 1);
27 fprintf('--- 开始 DPG 策略梯度优化 ---\n');
28 %% 4. 策略梯度更新循环
29 for iter = 1:num_iterations
30     % 假设状态分布探索充分: 从标准正态分布中采样一批状态  $x_k$ 
31     %  $x_k$  的维度为 (n_states x batch_size)
32     X_batch = randn(size(A, 1), batch_size);
33
34     grad_K_accum = zeros(size(K_dpg));
35
36     % 计算 Batch 内的经验策略梯度
37     for i = 1:batch_size
38         x_k = X_batch(:, i);
39
40         % 当前策略给出的动作  $u = -K * x_k$ 
41         u_k = -K_dpg * x_k;
42
43         % 1. 计算  $Q^*$  对  $u$  的梯度:  $\nabla_u Q^*(x_k, u)$ 
44         % 公式:  $2 * R * u + 2 * B' * P * (A * x_k + B * u)$ 
45         grad_u_Q = 2 * R * u_k + 2 * B' * P_star * (A * x_k + B * u_k);
46
47         % 2. 计算链式法则得到的策略梯度对  $K$  的偏导:  $\nabla_K J(K)$ 
48         % 因为我们要最小化代价值 (Minimize Cost), 所以直接使用你的推导公式:
49         %  $\nabla_K C_{sample} = - \nabla_u Q^* * x_k'$ 

```

```
50         grad_K_sample = -grad_u_Q * x_k';
51
52         % 累加梯度
53         grad_K_accum = grad_K_accum + grad_K_sample;
54     end
55
56     % 取 batch 的平均梯度
57     grad_K_avg = grad_K_accum / batch_size;
58
59     % 梯度下降更新 K (目标是最小化 Cost)
60     K_dpg = K_dpg - learning_rate * grad_K_avg;
61
62     % 记录当前 K_dpg 与真实 K_lqr 的 Frobenius 范数误差
63     error_history(iter) = norm(K_dpg - K_lqr, 'fro');
64 end
65 %% 5. 输出结果与可视化
66 fprintf('--- DPG 收敛结果 ---\n');
67 disp('最终 DPG 学习得到的增益 K_dpg:');
68 disp(K_dpg);
69 fprintf('K_dpg 与 K_lqr 的最终误差 (Frobenius Norm): %e\n\n',
    error_history(end));
70 % 绘制收敛曲线
71 figure('Name', 'DPG 收敛至 LQR', 'Color', 'w');
72 plot(1:num_iterations, error_history, 'LineWidth', 2, 'Color', [0 0.4470
    0.7410]);
73 xlabel('迭代次数 (Iterations)', 'FontSize', 12);
74 ylabel('误差 || K_{DPG} - K_{LQR} ||_F', 'FontSize', 12);
75 title('DPG 策略梯度下降收敛过程', 'FontSize', 14);
76 grid on;
```

对比

核心维度	DPG 视角 (无模型强化学习)	LQR 视角 (经典最优控制)	物理与数学联系 (对偶关系)
单步信号	奖励函数 $r_k = r(x_k, u_k) = x_k^T Q x_k + u_k^T R u_k$	单步代价 $c_k = x_k^T Q x_k + u_k^T R u_k$	符号相反： $r_k = -c_k$ 。RL 追求奖励最大化，LQR 惩罚状态偏差与控制能耗。
优化目标	累积回报 $J = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r_k \right]$ $Q(x_k, u_k) = r_k + \gamma Q(x_{k+1}, \mu(x_{k+1}))$	总代价 $J = \sum_{k=0}^{\infty} c_k$ $V^*(x_k) = c + V^*(x_{k+1})$	目标等价：最大化回报 $\iff$ 最小化总代价。LQR 通常不考虑折扣因子（即 $\gamma = 1$ ）。

状态评估(V 函数)	状态价值函数 $V^\pi(x)$ (从状态 x 开始, 遵循策略 π 的期望回报)	最优剩余代价 $V^*(x) = x^T P x$ (即李亚普诺夫函数)	符号相反且内涵统一: $VDPG = -V LQR$ 。LQR 的 $V(x)$ 必须严格递减 ($\Delta V < 0$) 以保证物理系统稳定 (收敛到原点)。
动作评估(Q 函数)	动作价值函数 $Q(x, u)$ (在状态 x 执行动作 u 后的总回报), 可以用神经网络, 也可以用LQR的解析表达式。	二次型 Q 函数 $q^*(x, u) = [x; u]^T H [x; u]$	优化依据: DPG 沿 $\nabla_u Q$ 进行梯度上升; LQR 通过令 $\frac{\partial q}{\partial u} = 0$ 得到解析极小值。 两者都是求一个最优的 K, 使得状态反馈控制 $u_K = -K x_k$ 达到使得总代价 J 最小 (或最大) 的目标。
策略/控制	确定性策略 $u = \mu(x; \theta)$ (通常用神经网络拟合)	状态反馈律 $u = -K x$, (K 由 Riccati 方程解出)	形式对应: LQR 的线性反馈是最优策略的解析解; DPG 利用神经网络逼近这一非线性映射。
核心算法	策略梯度 + TD 误差交替更新 Actor 和 Critic	求解 Riccati 方程代数计算出确定的 P 和 K	信息流向一致: 都是通过评估未来信息 (Q 函数 / P 矩阵) 来指导当前动作 (μ / K) 的更新。

注意：由于采用了确定性策略，对于DPG，导致了 $v(s_k) = q(s_k, a_k)$ 即某一个状态的价值等于该状态的动作值

critic不准确时的一些讨论

Critic（Q函数）的逼近误差会直接导致 Actor（策略）收敛到次优解。

在理想情况下，我们拥有完美的 Q^* ，其对应的矩阵是 P^* 。但在实际的 DPG 或 DDPG 算法中，Q 函数通常由神经网络或线性特征近似得到。假设我们学习到的 Q 函数存在微小偏差，等价于我们估计的 P 矩阵变为了：

$$P_{approx} = P^* + \Delta P$$

当把这个包含误差的 P_{approx} 代入你推导的策略梯度公式中时，梯度的静止点（使得 $\nabla_K J(K) = 0$ 的点）将发生偏移。解出来的偏置增益将变成：

$$K_{biased} = (R + B^T P_{approx} B)^{-1} B^T P_{approx} A$$

下面是演示这一现象的 MATLAB 脚本。我们将人为地给完美的 P^* 注入一点误差，观察策略梯度的收敛轨迹。

MATLAB 演示：Q 函数误差对策略收敛的影响

代码块

```
1 % dpq_error_demo.m
2 % 演示：当 Q 函数存在逼近误差时，DPG 策略梯度会收敛到次优解 (Biased Policy)
3 clear; clc; close all;
```

```

4
5 %% 1. 系统定义与真实 LQR 解
6 A = [1.1, 0.5;
7       0.0, 0.9];
8 B = [0.1;
9       1.0];
10 Q = eye(2);
11 R = 1;
12
13 % 真实的 LQR 最优解 (Baseline)
14 [P_star, ~, K_lqr] = dare(A, B, Q, R);
15
16 %% 2. 模拟 Q 函数的逼近误差 (Critic Error)
17 % 假设神经网络逼近 Q 函数时存在微小误差, 相当于 P 矩阵有偏差
18 % 我们引入一个对称的扰动矩阵 dP
19 dP = [0.2, 0.05;
20        0.05, 0.3];
21 P_approx = P_star + dP;
22
23 % 理论上, 带有这个误差的 Q 函数会导致策略收敛到 K_biased
24 K_biased_theory = inv(R + B' * P_approx * B) * B' * P_approx * A;
25
26 %% 3. DPG 策略梯度设置
27 rng(42);
28 K_dpg = rand(size(K_lqr)); % 随机初始化
29 learning_rate = 0.01;
30 num_iterations = 800;
31 batch_size = 200;
32
33 % 记录两种误差
34 error_to_lqr = zeros(num_iterations, 1); % 距离真实最优解的误差
35 error_to_biased = zeros(num_iterations, 1); % 距离带偏置理论解的误差
36
37 %% 4. 优化循环 (使用带误差的 P_approx)
38 for iter = 1:num_iterations
39     X_batch = randn(size(A, 1), batch_size);
40     grad_K_accum = zeros(size(K_dpg));
41
42     for i = 1:batch_size
43         x_k = X_batch(:, i);
44         u_k = -K_dpg * x_k;
45
46         % 注意: 这里使用的是 P_approx 而不是 P_star!
47         % 这模拟了 Actor 正在使用一个并不完美的 Critic 来指导梯度方向
48         grad_u_Q = 2 * R * u_k + 2 * B' * P_approx * (A * x_k + B * u_k);
49         grad_K_sample = -grad_u_Q * x_k';
50

```

```

51         grad_K_accum = grad_K_accum + grad_K_sample;
52     end
53
54     grad_K_avg = grad_K_accum / batch_size;
55     K_dpg = K_dpg - learning_rate * grad_K_avg;
56
57     % 记录收敛过程
58     error_to_lqr(iter) = norm(K_dpg - K_lqr, 'fro');
59     error_to_biased(iter) = norm(K_dpg - K_biased_theory, 'fro');
60 end
61
62 %% 5. 结果输出与可视化
63 fprintf('--- 收敛结果对比 ---\n');
64 disp('真实 LQR 最优增益 K_lqr:'); disp(K_lqr);
65 disp('理论偏置增益 K_biased_theory:'); disp(K_biased_theory);
66 disp('DPG 实际收敛增益 K_dpg:'); disp(K_dpg);
67
68 % 绘制收敛曲线
69 figure('Name', 'Critic 误差对策略收敛的影响', 'Color', 'w');
70 plot(1:num_iterations, error_to_lqr, 'LineWidth', 2, 'Color', [0.8500 0.3250
0.0980]); hold on;
71 plot(1:num_iterations, error_to_biased, 'LineWidth', 2, 'Color', [0 0.4470
0.7410], 'LineStyle', '--');
72 xlabel('迭代次数 (Iterations)', 'FontSize', 12);
73 ylabel('Frobenius 范数误差', 'FontSize', 12);
74 title('Q 函数逼近误差导致次优策略', 'FontSize', 14);
75 legend('|| K_{DPG} - K_{LQR} ||_F (距离真最优的偏差)', ...
76         '|| K_{DPG} - K_{Biased} ||_F (距离假最优的误差)', ...
77         'FontSize', 11);
78 grid on;

```

实验现象解析

当你运行这段代码时，你会观察到非常经典的**“策略收敛次优”**现象：

1. **无法达到真实最优解**：代表 $\| K_{\{DPG\}} - K_{\{LQR\}} \|$ 的橙色实线在下降到一定程度后会触底反弹或保持水平（Plateau），不会降为 0。这意味着哪怕你训练无限次，策略也永远到不了真正的 LQR 最优解。
2. **收敛到了“假想”最优**：代表 $\| K_{\{DPG\}} - K_{\{Biased\}} \|$ 的蓝色虚线却稳稳地收敛到了 0。这说明梯度下降算法本身没有错，Actor 非常尽职尽责地最大化了那个错误的 Q 函数，并准确地停在了错误 Q 函数的顶点上。

这就是为什么在强化学习中，**Critic 的准确性决定了 Actor 的上限**。如果 Q 函数学歪了，基于它计算出来的策略梯度就会把策略参数往错误的方向带。

2. 价值函数 Q 未知时（未收敛时），策略固定为线性状态反馈 $u = -Kx$ 的情况

当你不再已知真实的价值函数矩阵 P （即不知道环境模型 A, B ），你就正式跨入了无模型强化学习（Model-Free RL）的领域。

在已知模型时，你直接利用 P 计算出了 $\nabla_u Q$ 。但在无模型时，我们必须引入一个评价网络（Critic）来同时学习 Q 函数。这就是大名鼎鼎的 **Actor-Critic 架构**！

无模型 DPG 核心痛点与解决思路

在无模型设定下，代码将迎来两个极其关键的变化：

1. Critic 的引入（TD Learning）：

我们需要用一个矩阵 H 来参数化 Q 函数： $Q(x, u) = \begin{bmatrix} x \\ u \end{bmatrix}^T H \begin{bmatrix} x \\ u \end{bmatrix}$ 。

由于不知道真正的代价函数，我们只能让智能体与环境交互，收集 (x, u, c, x') 的数据，并利用时序差分（TD Error）和贝尔曼方程来拟合 H 。

2. 探索噪声（Exploration Noise）：

这是极其重要的一点！如果智能体一直严格执行 $u = -Kx$ ，那么 x 和 u 就是完全线性相关的。Critic 就无法区分到底是 x 导致了代价，还是 u 导致了代价（矩阵 H 会奇异化）。因此，我们必须在收集数据时加入**探索噪声**： $u = -Kx + \mathcal{N}(0, \sigma)$ 。

代码块

```
1  % dpq_lqr_modelfree.m
2  % 在未知环境模型 A, B 的情况下，仅依靠奖励(代价)函数 r 学习最优控制
3  clear; clc; close all;
4
5  %% 1. 环境定义 (对智能体是黑盒的)
6  A = [1.1, 0.5;
7       0.0, 0.9];
8  B = [0.1;
9       1.0];
10 Q = eye(2);
11 R = 1;
12 % 求解真实的 LQR 用于验证误差
13 [~, ~, K_lqr] = dare(A, B, Q, R);
14 fprintf('--- LQR 最优解 (Baseline) ---\n');
15 disp(K_lqr);
16
17 %% 2. Actor-Critic DPG 参数初始化
18 rng(42);
19 % Actor 初始化: 在无模型RL中, 通常需要一个能让系统基本不发散的初始策略(温启动)
```



```

20 K_actor = [1.0, 1.0];
21
22 % Critic 初始化:  $Q(x,u) = z' * H * z$ , 由于  $x$  是2维,  $u$  是1维,  $z$  是3维
23 %  $H$  是 3x3 的对称矩阵
24 H_critic = eye(3);
25
26 % 学习参数设定
27 alpha_critic = 0.05; % Critic 的学习率 (通常比 Actor 大, 需要先验估值准确)
28 alpha_actor = 0.005; % Actor 的学习率
29 num_epochs = 600; % 训练轮数
30 batch_size = 200; % 经验回放批次大小
31 gamma = 1.0; % LQR 是无折扣问题
32 sigma_noise = 0.5; % 探索噪声标准差 (极其关键!)
33
34 error_history = zeros(num_epochs, 1);
35 fprintf('--- 开始 Model-Free DPG 训练 ---\n');
36
37 %% 3. Actor-Critic 联合优化循环
38 for epoch = 1:num_epochs
39
40     % --- Step 1: 收集交互数据 (Experience Replay 思想) ---
41     X_batch = randn(2, batch_size);
42     U_batch = zeros(1, batch_size);
43     C_batch = zeros(1, batch_size);
44     X_next_batch = zeros(2, batch_size);
45
46     for i = 1:batch_size
47         x = X_batch(:, i);
48         % 执行带探索噪声的动作 (Behavior Policy)
49         u_noise = -K_actor * x + sigma_noise * randn();
50
51         % 与黑盒环境交互, 观察代价值和下一状态
52         c = x' * Q * x + u_noise' * R * u_noise;
53         x_next = A * x + B * u_noise;
54
55         % 存入 Buffer
56         U_batch(1, i) = u_noise;
57         C_batch(1, i) = c;
58         X_next_batch(:, i) = x_next;
59     end
60
61     % --- Step 2: 更新 Critic (Policy Evaluation, TD-Learning) ---
62     % Critic 为了拟合得更准, 在当前 batch 上多迭代几次
63     for iter_c = 1:10
64         grad_H_accum = zeros(3, 3);
65         for i = 1:batch_size
66             x = X_batch(:, i);

```

```

67         u = U_batch(1, i);
68         c = C_batch(1, i);
69         x_next = X_next_batch(:, i);
70
71         z = [x; u];
72
73         % 目标策略 (Target Policy): 评估当前 Actor 的纯策略 (无噪声)
74         u_next = -K_actor * x_next;
75         z_next = [x_next; u_next];
76
77         % 计算 TD 误差: 预测值 vs 目标值 (半梯度法, 不对目标求导)
78         Q_pred = z' * H_critic * z;
79         Q_target = c + gamma * (z_next' * H_critic * z_next);
80         delta = Q_pred - Q_target;
81
82         % 累加 Critic 梯度:  $\nabla_H \text{Loss} = \text{delta} * z * z'$ 
83         grad_H_accum = grad_H_accum + delta * (z * z');
84     end
85     % 梯度下降更新 H, 并强制对称化
86     H_critic = H_critic - alpha_critic * (grad_H_accum / batch_size);
87     H_critic = (H_critic + H_critic') / 2;
88 end
89
90 % --- Step 3: 更新 Actor (Policy Improvement, DPG) ---
91 grad_K_accum = zeros(1, 2);
92 % 从学习到的 H 矩阵中提取分块
93 H_uu = H_critic(3, 3);
94 H_ux = H_critic(3, 1:2);
95
96 for i = 1:batch_size
97     x = X_batch(:, i);
98     % DPG 计算梯度是在 当前 Actor 策略上进行的 (不带噪声)
99     u_policy = -K_actor * x;
100
101     % 从学到的 Critic 中计算动作梯度:  $\nabla_u Q(x, u)$ 
102     grad_u_Q = 2 * H_uu * u_policy + 2 * H_ux * x;
103
104     % 链式法则:  $\nabla_K J = - \nabla_u Q * x'$ 
105     grad_K_sample = -grad_u_Q * x';
106     grad_K_accum = grad_K_accum + grad_K_sample;
107 end
108
109 % 梯度下降更新 Actor (最小化代价值)
110 K_actor = K_actor - alpha_actor * (grad_K_accum / batch_size);
111
112 % 记录误差
113 error_history(epoch) = norm(K_actor - K_lqr, 'fro');

```

```

114 end
115
116 %% 4. 输出结果与可视化
117 fprintf('--- DPG (Model-Free) 收敛结果 ---\n');
118 disp('最终学习得到的增益 K_actor:');
119 disp(K_actor);
120 fprintf('与理论最优 K_lqr 的最终误差: %e\n\n', error_history(end));
121
122 figure('Name', 'Model-Free DPG 收敛曲线', 'Color', 'w');
123 plot(1:num_epochs, error_history, 'LineWidth', 2, 'Color', [0.8500 0.3250
0.0980]);
124 xlabel('迭代次数 (Epochs)', 'FontSize', 12);
125 ylabel('误差 || K_{Actor} - K_{LQR} ||_F', 'FontSize', 12);
126 title('无模型 DPG (Actor-Critic) 收敛过程', 'FontSize', 14);
127 grid on;

```

代码运行解析与进阶思考

当你运行这段代码后，你会发现虽然它不再使用真实的 P 矩阵，但 `K_actor` 依然像魔法一样收敛到了 `K_lqr`。

在这个过程中，有三个非常值得你注意的技术细节，它们直接对应着现代深度强化学习（如 DDPG / TD3）的底层逻辑：

- 两时间尺度（Two-Time-Scale）更新：**你会看到 Critic 的学习率 `alpha_critic` (0.05) 比 Actor 的学习率 `alpha_actor` (0.005) 大了整整 10 倍，并且在每一步都进行了内循环（10 次迭代）。这是因为在 Actor-Critic 中，**Critic 必须总是比 Actor 更新得快**。如果 Critic 还没有把 Q 值评估准，Actor 就急着根据错误的梯度去更新策略，整个系统瞬间就会崩溃。
- 经验回放（Experience Replay）的影子：**代码中使用了 `X_batch = randn(2, batch_size)` 来在状态空间中均匀采样，这其实模拟了 RL 中的打乱顺序的经验回放池，打破了连续时间步状态之间的高相关性，保证了 Critic 参数估计的无偏性。
- 异策略（Off-Policy）学习：**收集数据时用的是带噪声的**行为策略**（Behavior Policy），但在计算 TD 目标 `u_next` 和 DPG 梯度时，用的却是纯粹的**目标策略**（Target Policy）。这就是 DDPG 作为 Off-Policy 算法的数学体现。

将二次型的矩阵 H 和线性的增益 K 替换为深度神经网络（DNN），这段代码就变成了原汁原味的深度确定性策略梯度（DDPG）。

3. 价值函数 $Q(x, u) = \begin{bmatrix} x \\ u \end{bmatrix}^T H \begin{bmatrix} x \\ u \end{bmatrix}$ 确定时，采用深度神经网络取代策略 $u_k = -Kx_k$

代码块 `dpg_neural_network_actor.m`

```
2 % 使用神经网络作为 Actor ( $u = NN(x)$ ), 通过 DPG 学习 LQR 最优控制
3 clear; clc; close all;
4 %% 1. 系统定义与真实 LQR 解 (作为验证基准)
5 A_sys = [1.1, 0.5;
6          0.0, 0.9];
7 B_sys = [0.1;
8          1.0];
9 Q = eye(2);
10 R = 1;
11 % 求解真实的 LQR 解
12 [P_star, ~, K_lqr] = dare(A_sys, B_sys, Q, R);
13 %% 2. 神经网络 Actor 初始化 (1层隐藏层 MLP)
14 % 结构: Input (2) -> Hidden (16) -> Tanh -> Output (1)
15 n_states = 2;
16 n_hidden = 16;
17 n_actions = 1;
18 rng(42); % 固定随机种子
19 % 使用较小的随机值初始化权重, 防止初始动作过大导致系统发散
20 W1 = randn(n_hidden, n_states) * 0.1;
21 b1 = zeros(n_hidden, 1);
22 W2 = randn(n_actions, n_hidden) * 0.1;
23 b2 = zeros(n_actions, 1);
24 %% 3. DPG 训练超参数
25 learning_rate = 0.005;
26 num_iterations = 1500;
27 batch_size = 256;
28 % 生成一个固定的测试集, 用于评估神经网络策略与 LQR 策略的差距
29 X_test = randn(n_states, 100);
30 u_lqr_test = -K_lqr * X_test;
31 % 记录误差
32 mse_history = zeros(num_iterations, 1);
33 fprintf('--- 开始训练神经网络 Actor ---\n');
34 %% 4. 训练循环 (前向传播与反向传播)
35 for iter = 1:num_iterations
36     % 1. 采样状态 (保证状态空间的探索)
37     X_batch = randn(n_states, batch_size);
38
39     % ===== 前向传播 (Forward Pass) =====
40     Z1 = W1 * X_batch + b1; % 线性层 1
41     A1 = tanh(Z1); % 非线性激活函数 Tanh
42     U_batch = W2 * A1 + b2; % 线性输出层 (Actor 的动作 u)
43
44     % ===== 计算 Critic 梯度 =====
45     % 这里使用我们已知的真实最优 Q 函数 (Perfect Critic)
46     %  $grad\_U = \nabla_u Q^*(x, u) = 2Ru + 2B^T P (Ax + Bu)$ 
47     % 维度为 (n_actions x batch_size)
```

```

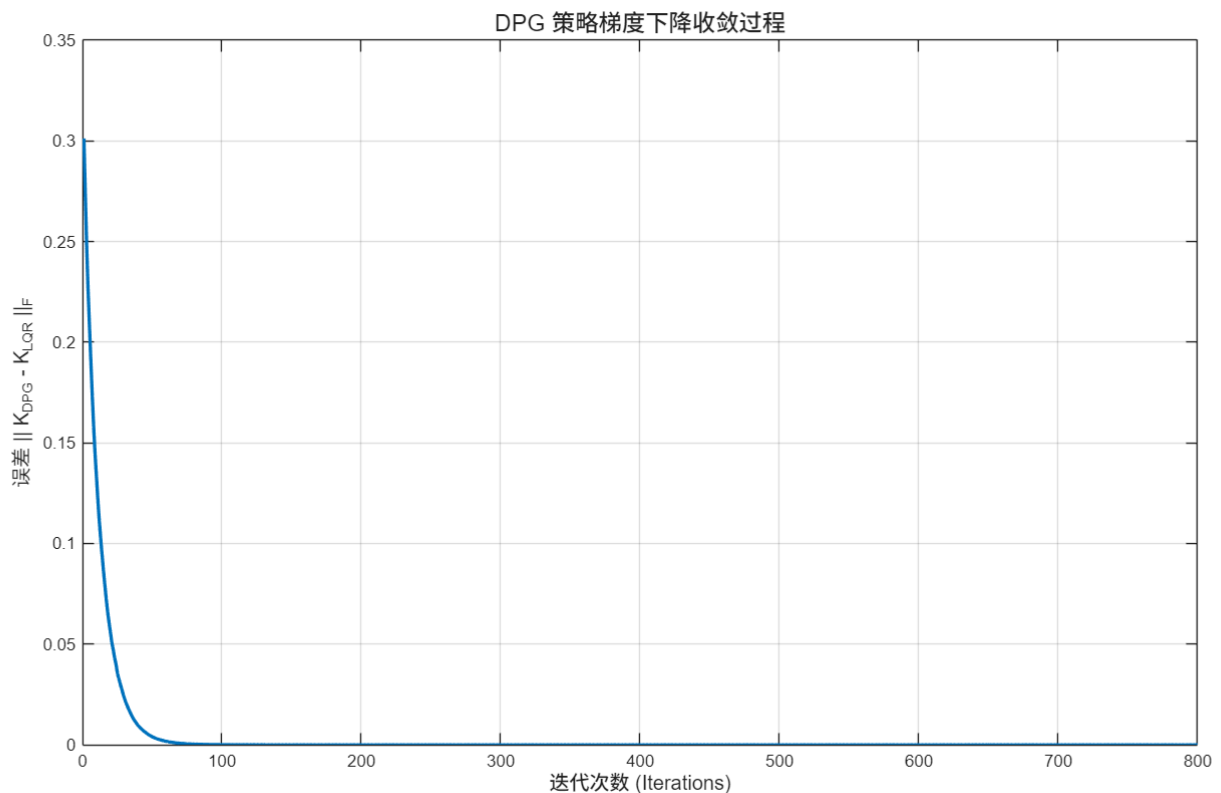
48     grad_U = 2 * R * U_batch + 2 * B_sys' * P_star * (A_sys * X_batch + B_sys
    * U_batch);
49
50     % ===== 反向传播 (Backward Pass / 链式法则) =====
51     % DPG 目标是最小化 Q 值，所以我们要顺着 grad_U 方向计算梯度并进行梯度下降
52
53     % 输出层梯度
54     grad_W2 = (grad_U * A1') / batch_size;           % dQ/dW2
55     grad_b2 = mean(grad_U, 2);                       % dQ/db2
56
57     % 隐藏层梯度 (反向传播过 Tanh)
58     grad_A1 = W2' * grad_U;                         % dQ/dA1
59     % Tanh 的导数是 (1 - tanh(x)^2)
60     grad_Z1 = grad_A1 .* (1 - A1.^2);               % dQ/dZ1
61
62     % 输入层梯度
63     grad_W1 = (grad_Z1 * X_batch') / batch_size; % dQ/dW1
64     grad_b1 = mean(grad_Z1, 2);                     % dQ/db1
65
66     % ===== 策略参数更新 (Gradient Descent) =====
67     W1 = W1 - learning_rate * grad_W1;
68     b1 = b1 - learning_rate * grad_b1;
69     W2 = W2 - learning_rate * grad_W2;
70     b2 = b2 - learning_rate * grad_b2;
71
72     % ===== 评估当前神经网络策略 =====
73     % 在测试集上比较 NN 输出的动作与 LQR 理论最优动作
74     A1_test = tanh(W1 * X_test + b1);
75     U_nn_test = W2 * A1_test + b2;
76     mse_history(iter) = mean((U_nn_test - u_lqr_test).^2, 'all');
77
78     if mod(iter, 300) == 0
79         fprintf('迭代 %d/%d, 测试集动作 MSE: %e\n', iter, num_iterations,
mse_history(iter));
80     end
81 end
82 fprintf('--- 训练完成 ---\n\n');
83 %% 5. 结果可视化: 对比 NN 输出与理论最优面
84 figure('Name', '神经网络 Actor vs LQR', 'Color', 'w', 'Position', [100, 100,
800, 350]);
85 % 子图 1: MSE 收敛曲线
86 subplot(1, 2, 1);
87 plot(1:num_iterations, mse_history, 'LineWidth', 2, 'Color', [0.4940, 0.1840,
0.5560]);
88 xlabel('迭代次数 (Iterations)', 'FontSize', 11);
89 ylabel('动作均方误差 (MSE)', 'FontSize', 11);
90 title('神经网络策略收敛过程', 'FontSize', 12);

```

```

91 set(gca, 'YScale', 'log'); % 使用对数坐标系更清晰
92 grid on;
93 % 子图 2: 在一个状态维度上画出控制面 (假设  $x_2 = 0$ )
94 subplot(1, 2, 2);
95 x1_range = linspace(-3, 3, 100);
96 x2_fixed = zeros(1, 100);
97 X_surf = [x1_range; x2_fixed];
98 % LQR 理论输出
99 u_lqr_surf = -K_lqr * X_surf;
100 % 训练好的神经网络输出
101 A1_surf = tanh(W1 * X_surf + b1);
102 u_nn_surf = W2 * A1_surf + b2;
103 plot(x1_range, u_lqr_surf, 'k--', 'LineWidth', 2); hold on;
104 plot(x1_range, u_nn_surf, 'r-', 'LineWidth', 1.5);
105 xlabel('状态  $x_1$  (当  $x_2 = 0$  时)', 'FontSize', 11);
106 ylabel('控制输入  $u$ ', 'FontSize', 11);
107 title('策略控制律对比', 'FontSize', 12);
108 legend('LQR 理论最优 (线性)', '神经网络 Actor (非线性)', 'Location', 'best');
109 grid on;

```



代码块

```

1 % dpd_dlnetwork_actor.m
2 % 使用 MATLAB Deep Learning Toolbox 实现 DPG 策略训练

```

```

3  clear; clc; close all;
4  %% 1. 系统定义与真实 LQR 解 (基准)
5  A_sys = [1.1, 0.5;
6           0.0, 0.9];
7  B_sys = [0.1;
8           1.0];
9  Q = eye(2);
10 R = 1;
11 % 求解真实的 LQR 解
12 [P_star, ~, K_lqr] = dare(A_sys, B_sys, Q, R);
13 %% 2. 定义神经网络 Actor (使用 MATLAB dlnetwork)
14 % 构建一个包含一层隐藏层的多层感知机 (MLP)
15 layers = [
16     featureInputLayer(2, 'Name', 'state')
17     fullyConnectedLayer(16, 'Name', 'fc1')
18     tanhLayer('Name', 'tanh1')
19     fullyConnectedLayer(1, 'Name', 'action')
20 ];
21 % 转换为未初始化的 dlnetwork 对象
22 actorNet = dlnetwork(layers);
23 %% 3. 训练超参数与数据准备
24 num_iterations = 800;
25 batch_size = 256;
26 learning_rate = 0.02;
27 % 用于 Adam 优化器的状态变量
28 trailingAvg = [];
29 trailingAvgSq = [];
30 % 固定的测试集, 用于评估收敛情况
31 X_test = randn(2, 100);
32 u_lqr_test = -K_lqr * X_test;
33 mse_history = zeros(num_iterations, 1);
34 fprintf('--- 开始基于 dlnetwork 的 DPG 训练 ---\n');
35 %% 4. 自定义训练循环
36 for iter = 1:num_iterations
37     % 1. 采样状态数据并转换为 dlarray
38     % 'CB' 表示维度顺序: Channel (特征维度) x Batch (批次大小)
39     X_batch = randn(2, batch_size);
40     dlX = dlarray(X_batch, 'CB');
41
42     % 2. 使用 dlfeval 计算损失和梯度
43     % 我们将真实系统参数 A_sys, B_sys, R, P_star 传给自定义损失函数
44     [loss, gradients] = dlfeval(@actorLoss, actorNet, dlX, A_sys, B_sys, R,
45                                 P_star);
46
47     % 3. 使用 Adam 优化器更新网络权重
48     [actorNet, trailingAvg, trailingAvgSq] = adamupdate(actorNet, gradients,
49 ...

```

```

48         trailingAvg, trailingAvgSq, iter, learning_rate);
49
50     % 4. 评估当前策略与 LQR 最优策略的差距
51     dlX_test = dldarray(X_test, 'CB');
52     dlU_test = predict(actorNet, dlX_test); % 评估模式前向传播
53     U_test = extractdata(dlU_test);          % 从 dldarray 提取数值
54
55     mse_history(iter) = mean((U_test - u_lqr_test).^2, 'all');
56
57     if mod(iter, 100) == 0
58         fprintf('迭代 %d/%d, Loss (Avg Q-value): %.4f, 动作 MSE: %e\n', ...
59             iter, num_observations, extractdata(loss), mse_history(iter));
60     end
61 end
62 fprintf('--- 训练完成 ---\n\n');
63 %% 5. 结果可视化
64 figure('Name', 'dldnetwork Actor vs LQR', 'Color', 'w', 'Position', [100, 100,
65     800, 350]);
66 subplot(1, 2, 1);
67 plot(1:num_observations, mse_history, 'LineWidth', 2, 'Color', [0.4940, 0.1840,
68     0.5560]);
69 xlabel('迭代次数 (Iterations)', 'FontSize', 11);
70 ylabel('动作均方误差 (MSE)', 'FontSize', 11);
71 title('dldnetwork 策略收敛过程', 'FontSize', 12);
72 set(gca, 'YScale', 'log');
73 grid on;
74 subplot(1, 2, 2);
75 x1_range = linspace(-3, 3, 100);
76 X_surf = [x1_range; zeros(1, 100)];
77 u_lqr_surf = -K_lqr * X_surf;
78 dlU_surf = predict(actorNet, dldarray(X_surf, 'CB'));
79 u_nn_surf = extractdata(dlU_surf);
80 plot(x1_range, u_lqr_surf, 'k--', 'LineWidth', 2); hold on;
81 plot(x1_range, u_nn_surf, 'r-', 'LineWidth', 1.5);
82 xlabel('状态 x_1 (当 x_2 = 0 时)', 'FontSize', 11);
83 ylabel('控制输入 u', 'FontSize', 11);
84 title('策略控制律对比', 'FontSize', 12);
85 legend('LQR 理论最优', 'dldnetwork Actor', 'Location', 'best');
86 grid on;
87 %% 6. 核心：自定义损失与梯度计算函数
88 %% 6. 核心：自定义损失与梯度计算函数（已修复）
89 function [loss, gradients] = actorLoss(actorNet, dlX, A, B, R, P_star)
90     % 前向传播获取动作
91     dlU = forward(actorNet, dlX);
92
93     % --- 核心修复区 ---
94     % 剥离维度标签 ('CB'), 以便与纯数字矩阵 A 和 B 进行标准的矩阵乘法

```



```

93     % stripdims 保留了变量的 dlarray 属性, 因此不会丢失自动求导的追踪
94     X_unformatted = stripdims(dlX);
95     U_unformatted = stripdims(dlU);
96
97     % 计算下一步状态:  $x_{k+1} = A \cdot x_k + B \cdot u_k$ 
98     dlX_next = A * X_unformatted + B * U_unformatted;
99
100    % 计算与控制相关的 Q 值 (作为要最小化的代价函数)
101    %  $Q(x, u) = x'Qx + u'Ru + x_{next}' * P * x_{next}$ 
102    cost_U = R * (U_unformatted .* U_unformatted); %  $u'Ru$ 
103
104    % 计算终端代价  $(Ax+Bu)' * P * (Ax+Bu)$ 
105    % 利用矩阵逐元素相乘 .* 和 sum 沿特征维度 (第一维) 求和
106    cost_NextState = sum(dlX_next .* (P_star * dlX_next), 1);
107
108    % 计算 Batch 的平均 Cost
109    loss = mean(cost_U + cost_NextState);
110
111    % 使用 dlgradient 自动计算 Loss 对网络所有可学习参数的梯度
112    gradients = dlgradient(loss, actorNet.Learnables);
113    end

```

代码块

```

1  % dpq_dlnetwork_all_dims_error.m
2  % 完善所有维度的误差对比与 3D 控制面可视化
3  clear; clc; close all;
4  %% 1. 系统定义与真实 LQR 解 (基准)
5  A_sys = [1.1, 0.5;
6           0.0, 0.9];
7  B_sys = [0.1;
8           1.0];
9  Q = eye(2);
10 R = 1;
11 [P_star, ~, K_lqr] = dare(A_sys, B_sys, Q, R);
12 fprintf('真实 LQR 增益 K_lqr = [%4f, %4f]\n', K_lqr(1), K_lqr(2));
13 %% 2. 定义神经网络 Actor
14 layers = [
15     featureInputLayer(2, 'Name', 'state')
16     fullyConnectedLayer(16, 'Name', 'fc1')
17     tanhLayer('Name', 'tanh1')
18     fullyConnectedLayer(1, 'Name', 'action')
19 ];
20 actorNet = dlnetwork(layers);
21 %% 3. 训练超参数与数据准备
22 num_iterations = 800;

```

```

23 batch_size = 256;
24 learning_rate = 0.02;
25 trailingAvg = []; trailingAvgSq = [];
26 % 固定的测试集, 包含广泛的状态空间分布
27 X_test = randn(2, 500) * 2;
28 u_lqr_test = -K_lqr * X_test;
29 % 误差追踪记录器
30 mse_history = zeros(num_iterations, 1);
31 k1_error_history = zeros(num_iterations, 1); % 维度 1 增益误差
32 k2_error_history = zeros(num_iterations, 1); % 维度 2 增益误差
33 fprintf('--- 开始训练, 追踪全维度误差 ---\n');
34 %% 4. 自定义训练循环
35 for iter = 1:num_iterations
36     % 采样批次
37     X_batch = randn(2, batch_size);
38     dLX = dlarray(X_batch, 'CB');
39
40     % 前向与反向传播 (调用已修复 stripdims 的损失函数)
41     [loss, gradients] = dlfeval(@actorLoss, actorNet, dLX, A_sys, B_sys, R,
42     P_star);
43
44     % Adam 更新
45     [actorNet, trailingAvg, trailingAvgSq] = adamupdate(actorNet, gradients,
46     ...
47     trailingAvg, trailingAvgSq, iter, learning_rate);
48
49     % --- 误差评估区 ---
50     dLX_test = dlarray(X_test, 'CB');
51     U_test = extractdata(predict(actorNet, dLX_test));
52
53     % 1. 总体 MSE 误差
54     mse_history(iter) = mean((U_test - u_lqr_test).^2, 'all');
55
56     % 2. 提取神经网络等效线性增益 (利用伪逆解最小二乘:  $U = -K * X \Rightarrow K = -U * \text{pinv}(X)$ )
57     K_nn_approx = -U_test * pinv(X_test);
58
59     % 3. 记录各个维度的增益误差
60     k1_error_history(iter) = abs(K_nn_approx(1) - K_lqr(1));
61     k2_error_history(iter) = abs(K_nn_approx(2) - K_lqr(2));
62
63     if mod(iter, 100) == 0
64         fprintf('迭代 %d: MSE = %.2e | k1误差 = %.2e | k2误差 = %.2e\n', ...
65         iter, mse_history(iter), k1_error_history(iter),
66         k2_error_history(iter));
67     end
68 end

```

```

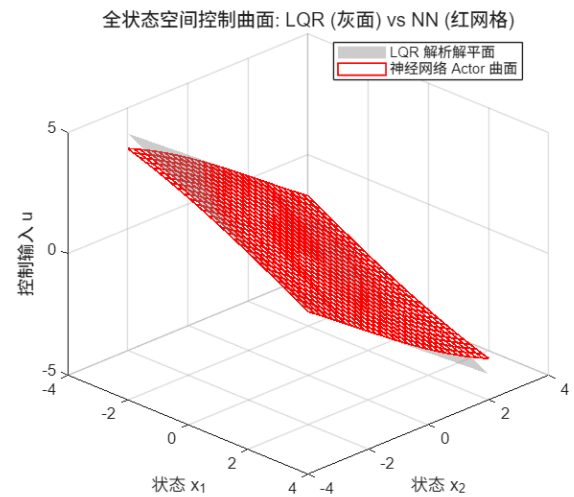
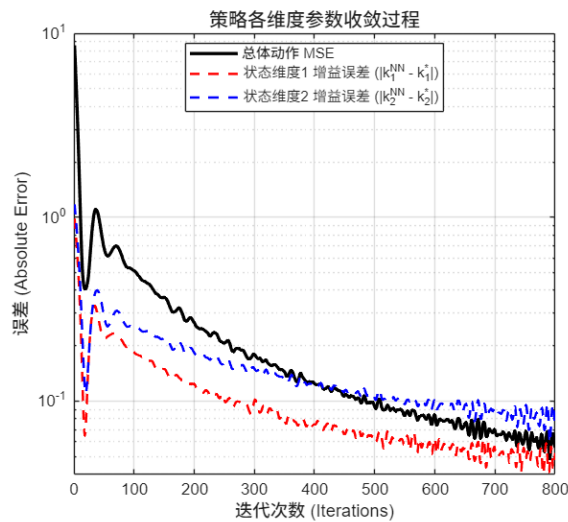
66 fprintf('--- 训练完成 ---\n\n');
67 %% 5. 结果可视化: 多维度收敛与 3D 控制曲面
68 figure('Name', 'DPG 全维度误差分析', 'Color', 'w', 'Position', [100, 100, 1200,
450]);
69 % 子图 1: 各维度误差收敛曲线
70 subplot(1, 2, 1);
71 plot(1:num_iterations, mse_history, 'k-', 'LineWidth', 2); hold on;
72 plot(1:num_iterations, k1_error_history, 'r--', 'LineWidth', 1.5);
73 plot(1:num_iterations, k2_error_history, 'b--', 'LineWidth', 1.5);
74 xlabel('迭代次数 (Iterations)', 'FontSize', 11);
75 ylabel('误差 (Absolute Error)', 'FontSize', 11);
76 title('策略各维度参数收敛过程', 'FontSize', 12);
77 set(gca, 'YScale', 'log'); % 对数坐标系显示微小误差
78 legend('总体动作 MSE', '状态维度1 增益误差 ( $|k_1^{NN} - k_1^*|$ )', ...
79        '状态维度2 增益误差 ( $|k_2^{NN} - k_2^*|$ )', 'Location', 'best');
80 grid on;
81 % 子图 2: 3D 状态-动作控制曲面全景对比
82 subplot(1, 2, 2);
83 [X1_grid, X2_grid] = meshgrid(linspace(-3, 3, 30), linspace(-3, 3, 30));
84 X_flat = [X1_grid(:)'; X2_grid(:)'];
85 % LQR 理论平面
86 U_lqr_flat = -K_lqr * X_flat;
87 U_lqr_grid = reshape(U_lqr_flat, size(X1_grid));
88 % 神经网络拟合曲面
89 dX_flat = dlmatrix(X_flat, 'CB');
90 U_nn_flat = extractdata(predict(actorNet, dX_flat));
91 U_nn_grid = reshape(U_nn_flat, size(X1_grid));
92 % 绘制 LQR 平面 (半透明灰色)
93 surf(X1_grid, X2_grid, U_lqr_grid, 'FaceColor', [0.5 0.5 0.5], 'FaceAlpha',
0.4, 'EdgeColor', 'none');
94 hold on;
95 % 绘制 NN 曲面 (彩色线框)
96 mesh(X1_grid, X2_grid, U_nn_grid, 'EdgeColor', 'r', 'LineWidth', 1);
97 xlabel('状态 x_1', 'FontSize', 11);
98 ylabel('状态 x_2', 'FontSize', 11);
99 zlabel('控制输入 u', 'FontSize', 11);
100 title('全状态空间控制曲面: LQR (灰面) vs NN (红网格)', 'FontSize', 12);
101 legend('LQR 解析解平面', '神经网络 Actor 曲面', 'Location', 'best');
102 view(45, 30); % 调整 3D 视角
103 grid on;
104 %% 6. 自定义损失与梯度计算函数 (含 stripdims 修复)
105 function [loss, gradients] = actorLoss(actorNet, dX, A, B, R, P_star)
106     dU = forward(actorNet, dX);
107
108     % 剥离维度标签进行矩阵运算
109     X_unformatted = stripdims(dX);
110     U_unformatted = stripdims(dU);

```

```

111
112     dlX_next = A * X_unformatted + B * U_unformatted;
113
114     cost_U = R * (U_unformatted .* U_unformatted);
115     cost_NextState = sum(dlX_next .* (P_star * dlX_next), 1);
116     loss = mean(cost_U + cost_NextState);
117
118     gradients = dlgradient(loss, actorNet.Learnables);
119 end

```



4. 采用深度神经网络取代价值函数 Q 和策略 u 的情况（DPG的雏形）